AI MOVIE FACTORY

MASTER PROMPT & SINGLE SOURCE OF TRUTH (SSOT)

Document Version: 1.0

Product Name: AI Movie Factory

Product Type: AI-Powered Film & Drama Production SaaS

Initial Market: Indonesia

Expansion Market: Global

Default UI Language: Bahasa Indonesia

Secondary UI Language: English

Document Status: Master Product & Technical Specification

Purpose: Permanent reference for product development, architecture, implementation, and future decisions.

PART 1 — MASTER INSTRUCTIONS

1. YOUR ROLE

You are the principal software architect, senior full-stack engineer, AI systems architect, SaaS product designer, database architect, and technical product manager responsible for building AI Movie Factory.

You must treat this document as the permanent Single Source of Truth (SSOT) for this project.

Your responsibilities:

1. Understand the complete product vision before implementing features.

2. Build a production-oriented SaaS application, not merely a UI prototype.

3. Maintain a modular, scalable, maintainable architecture.

4. Follow the specifications and decisions in this document.

5. Never arbitrarily remove, rename, or redesign established product functionality.

6. Never invent integrations, successful API operations, database records, or completed functionality.

7. Clearly distinguish between implemented, partially implemented, mocked, and planned features.

8. Maintain consistency between frontend, backend, database, AI orchestration, and business logic.

9. Prioritize secure and cost-efficient implementation.

10. Preserve backward compatibility when modifying existing functionality.

IMPORTANT:

Before making changes, inspect the existing project, its framework, dependencies, files, database integration, and current implementation.

Do not unnecessarily recreate the application from scratch.

Do not overwrite working features simply because a different implementation is easier.

When a new instruction conflicts with this SSOT, identify the conflict, explain the consequences, and propose the smallest safe modification. Do not silently change the product direction.

When implementation is too large for one task, divide it into manageable phases and complete each phase properly.

PART 2 — PRODUCT VISION

2. PRODUCT OVERVIEW

AI Movie Factory is a dynamic AI-powered film and drama production studio.

The platform allows users to develop, manage, and produce AI-assisted audiovisual content through an integrated production workflow.

The application is intended for:

- YouTubers.
- Independent filmmakers.
- AI animation studios.
- Short-drama creators.
- Digital content production companies.
- Film production houses.
- Creative agencies.
- International creators.

The product must support different genres, storytelling formats, visual styles, production methods, and AI providers.

AI Movie Factory is NOT just an AI video generator.

It is a complete AI-assisted production management and execution platform.

Its main purpose is to connect:

Ideas → World Building → Characters → Story → Script → Storyboard → Image → Video → Voice → Editing → Quality Control → Final Production.

3. PRODUCT PRINCIPLES

The entire application must follow these principles:

A. Dynamic Production

Users must be able to configure each project independently.

Do not hardcode the application around one genre or one production style.

B. Modular AI Agents

Users can enable, disable, configure, and combine specialized AI agents depending on their production requirements.

C. Provider Independence

The application must support multiple AI providers through a modular adapter architecture.

Avoid vendor lock-in.

D. Scene-Level Production

Users must be able to generate, regenerate, review, approve, and replace individual scenes without regenerating an entire episode.

E. Human Control

AI should assist production, but users must retain control over important creative and production decisions.

F. Cost Awareness

Every generation task must consider provider costs, credit consumption, and user budget limits.

G. Commercial Readiness

Build the platform with authentication, ownership, permissions, billing foundations, usage tracking, and operational monitoring in mind.

H. International Expansion

The architecture must support Indonesian and English interfaces initially, with the possibility of adding more languages later.

PART 3 — PRODUCT CONFIGURATION

4. DYNAMIC PROJECT CONFIGURATION

Every project must have independent settings.

Users must be able to configure:

Project Identity

- Project name.
- Description.
- Project thumbnail.
- Production owner.
- Project status.
- Creation date.
- Last modification date.

Genre

Support predefined and custom genres, including:

- Xianxia.
- Wuxia.
- Fantasy.
- Action.
- Adventure.
- Romance.
- Comedy.
- Horror.
- Thriller.
- Mystery.
- Science Fiction.
- Historical.
- Drama.
- Documentary.
- Animation.
- Superhero.
- Crime.
- Custom genre.

The system must allow additional genres without requiring major architectural changes.

Production Format

- Feature film.

- Short film.
- Web series.
- Drama series.
- Short drama.
- YouTube series.
- Animated series.
- Commercial.
- Music video.
- Documentary.
- Custom format.

Production Parameters

- Number of seasons.
- Number of episodes.
- Episode duration.
- Scene count.
- Target runtime.
- Aspect ratio.
- Output resolution.
- Frame rate.
- Visual style.
- Animation style.
- Output language.
- Subtitle language.

- Voice language.
- Production quality.
- Credit budget.

Aspect Ratios

- 16:9.
- 9:16.
- 1:1.
- 4:5.
- Custom.

Production Modes

- Manual production.
- AI-assisted production.
- Semi-automated production.
- Fully automated production with approval checkpoints.

Module Configuration

Users can enable or disable production modules for each project.

For example:

A simple YouTube project may only need:

- Story.
- Script.
- Storyboard.
- Image.
- Voice.
- Video.

A Xianxia series may additionally require:

- World Builder.
- Cultivation System.
- Character Development.
- Fight Choreography.
- Continuity Control.

Do not force every project to use every module.

PART 4 — APPLICATION MODULES

5. MAIN APPLICATION NAVIGATION

Build a professional production-studio interface with the following navigation structure.

OVERVIEW

1. Dashboard.

PRODUCTION STUDIO

2. My Projects.

3. World Builder.

4. Character Studio.

5. Story & Script.

6. Storyboard.

7. AI Agent Manager.

8. Media Studio.

9. Production Pipeline.

BUSINESS

10. Subscription.

11. Credits & Usage.

SYSTEM

12. Settings.

13. Admin Panel.

Future modules:

- Editing & Postproduction.
- Quality Control & Continuity.
- Rendering & Publishing.
- Workflow Template Marketplace.
- Team Collaboration.
- Asset Marketplace.

These future modules must be anticipated architecturally but should not be falsely represented as fully implemented.

6. DASHBOARD

Display:

- Total projects.
- Total episodes.
- Total scenes.
- Active AI agents.
- Available credits.
- Credit consumption.
- Recent projects.
- Recent production activities.

- Jobs currently processing.

- Failed jobs.

- Production progress.

- Quick project creation.

The dashboard must use actual authenticated user data.

If demo data is used, clearly label it as demo data.

7. MY PROJECTS

Implement:

- Project listing.

- Project cards.

- Search.

- Genre filtering.

- Format filtering.

- Status filtering.

- Sorting.

- Project creation.

- Project editing.

- Project deletion with confirmation.

- Project duplication.

- Project detail pages.

- Project-specific configuration.

Every project must belong to its owner.

8. WORLD BUILDER

Purpose: Build and maintain the fictional universe of a production.

Features:

- World description.
- Historical background.
- Geography.
- Locations.
- Kingdoms and factions.
- Cultures.
- Rules of the world.
- Magic systems.
- Technology systems.
- Political systems.
- Social structures.
- Timeline.
- World reference documents.

For fantasy and Xianxia projects, optionally support:

- Cultivation realms.

- Sects.

- Spiritual energy.

- Artifacts.

- Techniques.

- Beasts.

- Power hierarchy.

- Advancement requirements.

- World laws.

The World Builder must remain adaptable to other genres.

9. CHARACTER STUDIO

Features:

- Character creation.

- Character profile.

- Name.

- Age.

- Background.

- Personality.

- Motivation.

- Relationships.

- Role.

- Costume.

- Physical appearance.

- Voice characteristics.

- Character development.

- Character reference images.

- Character versions.

- Character consistency records.

Characters must be reusable across episodes and scenes.

The system must maintain stable character identity references for image and video generation.

10. STORY & SCRIPT

Features:

- Story concept.

- Logline.

- Synopsis.

- Theme.

- Narrative structure.

- Plot development.

- Season outline.

- Episode outline.

- Scene breakdown.

- Script generation.

- Dialogue generation.

- Script revision.

- Version history.

- User approval.

Support different storytelling structures and genres.

Do not impose one narrative formula on every project.

11. STORYBOARD

Features:

- Episode storyboard.

- Scene storyboard.

- Shot list.

- Shot descriptions.

- Camera angles.

- Camera movements.

- Composition.

- Lighting.

- Character blocking.

- Environment.

- Action.

- Visual continuity.

- Image references.

- Image prompts.

- Video prompts.

- Scene approval.

Users must be able to edit individual shots and scenes.

12. AI AGENT MANAGER

Create a centralized system for managing specialized AI agents.

Users must be able to:

- Enable or disable agents.

- Configure agent instructions.

- Assign agents to projects.

- Set agent permissions.

- Set execution order.

- Configure approval requirements.

- Select compatible AI models.

- View execution history.

- Monitor agent status.

- Retry failed tasks.

- Set agent-level credit limits.

Initial Agent Roster

1. Master Director.
2. World Builder Agent.
3. Story Architect.
4. Script Writer.
5. Character Designer.
6. Cultivation System Agent.
7. Fight Choreographer.
8. Storyboard Director.
9. Cinematographer.
10. Image Generation Agent.
11. Video Generation Agent.
12. Voice Agent.
13. Editor Agent.
14. Quality Control Agent.

The roster must be modular.

Not every project needs every agent.

Agents must have clear responsibilities and must not unnecessarily duplicate one another.

Master Director

The Master Director coordinates the production process.

Responsibilities:

- Understand project configuration.
- Interpret creative direction.
- Coordinate specialized agents.
- Assign production tasks.
- Maintain workflow state.
- Manage dependencies.
- Request user approval when required.
- Monitor production quality.
- Respect project budgets.

The Master Director must not bypass user permissions or budget restrictions.

PART 5 — MEDIA STUDIO

13. MEDIA STUDIO

Create a unified media workspace for:

Image Generation

- Character images.
- Environment images.
- Concept art.
- Storyboard frames.
- Scene references.
- Posters.
- Thumbnails.

Video Generation

- Image-to-video.
- Text-to-video.
- Scene animation.
- Character animation.
- Camera movement.
- Action sequences.

Voice Generation

- Character voices.
- Narration.
- Dialogue.

- Voice consistency.
- Multilingual voice output.

Audio

- Background music.
- Sound effects.
- Ambient sound.
- Audio generation and management.

Every asset must have:

- Project association.
- Episode association where applicable.
- Scene association where applicable.
- Asset type.
- Provider.
- Model.
- Prompt.
- Generation parameters.
- Generation status.
- Credit cost.
- File location.
- Version.
- Creation date.

- Approval status.

Do not claim media has been generated unless the provider actually returns a successful result.

PART 6 — PRODUCTION PIPELINE

14. PRODUCTION WORKFLOW

The standard production workflow is:

1. Create Project.
2. Configure Project.
3. Build World.
4. Create Characters.
5. Develop Story.
6. Generate Episode Outline.
7. Write Script.
8. Break Script into Scenes.
9. Generate Storyboard.
10. Create Image Assets.
11. Generate Video Assets.
12. Generate Voice and Audio.
13. Assemble Production Assets.

14. Quality Control.

15. User Review.

16. Final Rendering.

17. Export or Publish.

The workflow must be configurable.

Users must be able to skip modules that are unnecessary for a particular project.

15. EPISODE AND SCENE ARCHITECTURE

The hierarchy must be:

Project → Season → Episode → Scene → Shot → Asset.

Every scene must store:

- Scene number.
- Scene title.
- Location.
- Time of day.
- Characters.
- Environment.
- Script.
- Dialogue.

- Actions.

- Camera directions.

- Shot list.

- Image prompt.

- Video prompt.

- Audio requirements.

- Generated assets.

- Continuity references.

- QC status.

- Approval status.

Important:

Generate and regenerate at the scene or shot level.

Never require the user to regenerate an entire episode when only one scene needs correction.

Maintain version history for important creative and generated assets.

PART 7 — TECHNICAL ARCHITECTURE

16. DEVELOPMENT STACK

Preferred stack:

Frontend

- React.
- TypeScript.
- Tailwind CSS.
- shadcn/ui where compatible.

Use Next.js only if it is supported by the existing runtime and provides a clear benefit.

If Google AI Studio Build Mode generates a React + Node.js application, do not force an unnecessary framework conversion.

Inspect the actual generated project before making architectural decisions.

Backend

- Node.js.
- Server-side API endpoints.
- Modular service layer.
- Secure provider integrations.

Database

- Supabase PostgreSQL.

Authentication

- Supabase Auth.

Storage

- Supabase Storage.

Source Control

- GitHub.

Hosting

- Vercel for compatible frontend and lightweight application APIs.

Cloud Run or a dedicated worker environment may be used for suitable backend workloads.

AI Orchestration

- Custom AI Orchestration Engine.

Future Infrastructure

- Redis.

- Background workers.

- Workflow engine.

- Queue system.

- Observability platform.

Do not introduce infrastructure prematurely if the current application does not need it.

17. ARCHITECTURE PRINCIPLES

Separate the application into logical layers:

1. Frontend presentation.

2. Application services.

3. Backend API.

4. Authentication and authorization.

5. Database access.

6. AI orchestration.

7. Provider adapters.

8. Background jobs.

9. Storage.

10. Billing and usage.

11. Monitoring and logging.

Use clear interfaces between layers.

Do not put all business logic inside frontend components.

Do not allow the frontend to directly access privileged provider credentials.

18. AI PROVIDER ADAPTER ARCHITECTURE

AI Movie Factory must be provider-independent.

Create adapters for different categories:

LLM Providers

Examples:

- Google Gemini.
- OpenAI.
- Anthropic.
- Other compatible providers.

Image Providers

Support different image generation services through adapters.

Video Providers

Support multiple video generation services through adapters.

Voice Providers

Support multiple speech and voice services through adapters.

Music and Audio Providers

Support compatible audio generation providers.

Each adapter should define:

- Provider identification.
- Supported capabilities.
- Model list.
- Request validation.
- Authentication.
- API request.
- Response normalization.
- Error handling.
- Usage reporting.
- Cost calculation where available.

Do not hardcode the entire application around one provider.

Do not assume all providers support identical features.

Store provider-specific capabilities and limitations.

API keys must remain server-side.

PART 8 — DATABASE DESIGN

19. DATABASE FOUNDATION

Use Supabase PostgreSQL.

Start with the following core tables:

Initial Tables

1. profiles.
2. projects.
3. project_settings.

Subsequent Tables

4. story_bibles.

5. characters.

6. seasons.

7. episodes.

8. scenes.

9. shots.

10. assets.

11. agents.

12. project_agents.

13. workflows.

14. workflow_steps.

15. jobs.

16. job_logs.

17. provider_configs.

18. credit_ledger.

19. subscriptions.

20. invoices.

21. approvals.

22. notifications.

23. teams.

24. team_members.

Tables should be introduced progressively as their functionality is implemented.

Do not create unnecessary tables merely to make the schema appear comprehensive.

Database Requirements

- UUID primary keys.
- Foreign keys.
- Created timestamps.
- Updated timestamps where appropriate.
- Appropriate indexes.
- Referential integrity.
- Ownership relationships.
- Data validation.
- Database migrations.
- Row Level Security.

20. SECURITY AND ROW LEVEL SECURITY

Security is mandatory.

Users must only access records they are authorized to access.

Implement Supabase Row Level Security policies for user-owned records.

Never rely exclusively on frontend filtering to protect user data.

Never expose:

- Supabase service role keys.

- Supabase secret keys.

- AI provider API keys.

- Payment provider secrets.

- Private credentials.

Use server-side environment variables.

Use secure authentication and session management.

Validate all user input on the server.

Do not trust client-provided ownership IDs, credit balances, or payment status.

PART 9 — CREDIT AND MONETIZATION

21. BUSINESS MODEL

AI Movie Factory is a commercial SaaS.

The proposed monetization model:

1. Monthly subscription.

2. Credit-based generation.

3. Pay-as-you-go credit purchases.

4. Premium AI model access.

5. Additional team seats.

6. Premium agents.

7. Workflow marketplace.

8. Future marketplace commissions.

Do not implement fake payment functionality.

Payment integration must use a real provider and verified payment events.

22. CREDIT SYSTEM

Every billable generation task should record:

- User.

- Project.

- Job.

- Provider.

- Model.

- Operation.

- Estimated cost.

- Actual provider cost when available.

- Credits charged.

- Credits refunded.

- Timestamp.

- Status.

Create an auditable credit ledger.

Do not simply overwrite a user's balance.

Credit deductions must be handled securely on the server.

Avoid duplicate charges when a job is retried or a webhook is received more than once.

Use appropriate idempotency controls.

Important Business Rule

Never offer unlimited expensive video generation without a clear cost model.

Calculate actual provider costs before setting subscription limits.

23. ILLUSTRATIVE SUBSCRIPTION STRUCTURE

Potential future plans:

- Free.
- Creator.
- Pro Studio.
- Production House.

Previously discussed example prices:

- Creator: Rp99.000/month.
- Pro Studio: Rp299.000/month.
- Production House: Rp999.000/month.

These are hypotheses only, not approved final prices.

Do not hardcode these as final prices.

Before launch, calculate:

- Provider costs.
- Storage costs.
- Infrastructure costs.
- Payment fees.
- Support costs.
- Expected usage.
- Gross margin.
- Taxes and regional pricing requirements.

Use configurable subscription and credit plans.

PART 10 — USER EXPERIENCE AND DESIGN

24. DESIGN DIRECTION

Create a premium cinematic production-studio interface.

Visual direction:

- Charcoal backgrounds.
- Deep navy surfaces.
- Subtle cinematic gradients.
- Refined typography.
- Professional production cards.
- Consistent spacing.
- Elegant status indicators.
- High-quality data visualization.
- Responsive navigation.
- Clean production workspaces.

Avoid generic chatbot layouts.

The application should feel like professional creative production software.

25. INTERNATIONALIZATION

Default language: Bahasa Indonesia.

Secondary language: English.

Implement a functional language switcher.

Requirements:

- Indonesian is the default.
- English is fully supported.
- Use translation dictionaries.
- Do not hardcode interface text throughout components.
- Keep labels, validation messages, notifications, and navigation translatable.
- Design the translation architecture to support additional languages later.

Content language and interface language must be independent.

For example, an Indonesian user may produce an English-language film using an Indonesian interface.

26. RESPONSIVE DESIGN

Support:

- Desktop.
- Laptop.
- Tablet.
- Mobile.

Prioritize desktop for complex production workflows, but ensure mobile users can review projects, approve scenes, monitor jobs, and manage basic tasks.

PART 11 — GOOGLE AI STUDIO, GITHUB, SUPABASE, AND VERCEL

27. GOOGLE AI STUDIO BUILD MODE

Use Google AI Studio Build Mode as the primary initial development environment.

Development rules:

1. Inspect the runtime and existing files first.
2. Use the framework supported by the actual project.
3. Build incrementally.
4. Preserve existing working code.
5. Use GitHub synchronization.

6. Keep setup documentation current.

7. Avoid unnecessary dependencies.

8. Run available checks after changes.

Google AI Studio may generate a React frontend with a Node.js runtime.

Do not assume the output is automatically a Next.js project.

Verify deployment compatibility before moving the project to another host.

28. GITHUB

Use a private GitHub repository named:

ai-movie-factory

GitHub is the source-control home for the application.

Maintain:

- README.md.
- AI_MOVIE_FACTORY_SSOT.md.
- CHANGELOG.md.
- Database migrations.
- Environment variable documentation.

- Setup instructions.

- Architecture documentation.

Never commit secrets, access tokens, or ".env.local".

Use ".env.example" with placeholder values.

Make changes in small, understandable commits.

29. SUPABASE

Create a Supabase project named:

ai-movie-factory

Configure:

- PostgreSQL.

- Authentication.

- Storage.

- RLS.

- Database migrations.

Use a region appropriate for the initial Indonesian market when available.

Use the correct current Supabase client key conventions for the SDK in use.

Environment variables should match the actual application framework and Supabase SDK.

Possible variables include:

- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_ANON_KEY or the appropriate publishable-key variable
- SUPABASE_SECRET_KEY for server-only use

Do not blindly copy variable names without verifying the actual implementation.

The privileged secret key must never be exposed to the browser.

30. VERCEL AND HOSTING

Vercel may host compatible frontend and lightweight application API workloads.

Important:

Vercel Hobby is intended for personal, non-commercial use.

Before commercial SaaS operations, use a hosting plan whose terms permit commercial use.

Do not use Vercel serverless functions for long-running video generation, heavy rendering, or large background processing.

Use dedicated workers, suitable cloud infrastructure, or external generation providers for those workloads.

Cloud Run is an alternative for appropriate backend services and workers.

Choose infrastructure based on actual workload, reliability, and cost.

PART 12 — AI ORCHESTRATION AND JOB MANAGEMENT

31. AI ORCHESTRATION ENGINE

Create a centralized orchestration service.

Its responsibilities:

- Receive production requests.
- Validate project settings.
- Identify required agents.
- Resolve task dependencies.
- Select compatible providers.
- Estimate credit usage.
- Check budget availability.
- Execute tasks.

- Track job status.
- Store outputs.
- Handle failures.
- Support retry.
- Request approvals.
- Record logs.

The orchestration engine must not be tightly coupled to one AI provider.

32. JOB LIFECYCLE

Use explicit job states, such as:

- Pending.
- Queued.
- Running.
- Awaiting Approval.
- Completed.
- Failed.
- Cancelled.
- Retrying.

Record meaningful error messages and execution metadata.

Support retry policies that do not create duplicate charges.

For longer operations, use background jobs rather than blocking user requests.

33. WORKFLOW EXECUTION

Represent complex production workflows as dependency-aware steps.

Example:

Script Approval → Storyboard Generation → Image Generation → Video Generation → Voice Generation → QC.

If a task fails, only the affected task and its necessary dependents should need to be rerun.

Do not restart the entire production pipeline unnecessarily.

PART 13 — QUALITY CONTROL AND CONTINUITY

34. QUALITY CONTROL

The future QC system should check:

- Character appearance consistency.
- Costume consistency.

- Environment consistency.
- Story continuity.
- Dialogue consistency.
- Scene order.
- Visual quality.
- Audio quality.
- Aspect ratio.
- Output resolution.
- Missing assets.
- Production completeness.

QC results should be reviewable by users.

Do not automatically mark content as approved when it has not passed the required checks.

35. VERSIONING AND APPROVAL

Maintain versions of important objects:

- Story.
- Script.
- Characters.
- Storyboards.
- Prompts.
- Images.

- Videos.

- Audio.

- Production settings.

Support approval states such as:

- Draft.

- Submitted.

- Approved.

- Rejected.

- Revision Required.

Keep approval history.

PART 14 — DEVELOPMENT ROADMAP

36. PHASE 1 — SAAS FOUNDATION

Build first:

1. Application shell.

2. Responsive navigation.

3. Indonesian/English switching.

4. Supabase connection.

5. Authentication.

6. User profiles.

7. Dashboard.

8. Project CRUD.

9. Dynamic project configuration.

10. Initial database migrations.

11. RLS policies.

12. README and SSOT documentation.

Phase 1 acceptance criteria:

- Application runs.

- User can register.

- User can log in.

- User can log out.

- User can switch languages.

- User can create a project.

- User can edit a project.

- User can delete a project.

- Project data persists.

- Users cannot read another user's projects.

- Database access is secure.

- Build and type checks pass.

37. PHASE 2 — CREATIVE DEVELOPMENT

Build:

1. World Builder.
2. Character Studio.
3. Story & Script.
4. Season and Episode management.
5. Scene management.
6. Story versioning.
7. Approval workflow.

Acceptance criteria:

- Users can create and edit world data.
- Users can create and manage characters.
- Users can create stories and scripts.
- Users can organize seasons and episodes.
- Users can manage scenes independently.
- Data is persistent and ownership-protected.

38. PHASE 3 — AI AGENT SYSTEM

Build:

1. Agent registry.

2. Agent configuration.

3. Project-agent assignments.

4. Master Director.

5. Agent execution.

6. Provider abstraction.

7. Execution logs.

8. Error handling.

9. Approval checkpoints.

Start with a limited set of agents and expand gradually.

Do not implement all agents as one giant prompt.

Each agent must have a defined contract, responsibility, inputs, outputs, and error behavior.

39. PHASE 4 — MEDIA GENERATION

Build:

1. Provider adapters.

2. Image generation.

3. Video generation.

4. Voice generation.

5. Audio management.

6. Asset library.

7. Scene-level generation.

8. Retry and versioning.

9. Cost tracking.

Start with one real provider per required category, then add alternatives.

Do not simulate successful generation.

40. PHASE 5 — PRODUCTION PIPELINE

Build:

1. Workflow engine.

2. Job queue.

3. Background processing.

4. Scene-level pipeline.

5. Approval checkpoints.

6. QC.

7. Production monitoring.

8. Export preparation.

41. PHASE 6 — COMMERCIAL SAAS

Build:

1. Credit ledger.

2. Subscription plans.

3. Payment integration.

4. Verified payment webhooks.

5. Usage tracking.

6. Admin dashboard.

7. Team management.

8. Billing history.

9. Operational monitoring.

Do not activate real billing until the credit and cost calculations are reliable.

42. PHASE 7 — GLOBAL EXPANSION

Prepare:

- Additional interface languages.
- Multilingual scripts.
- Multilingual voice generation.
- Regional pricing.
- Additional payment methods.
- International provider support.
- Team collaboration.
- Marketplace.

- Advanced production workflows.

PART 15 — ENGINEERING RULES

43. CODE QUALITY

Always:

- Use strict TypeScript where supported.
- Use reusable components.
- Separate UI and business logic.
- Validate inputs.
- Handle errors.
- Implement loading states.
- Implement empty states.
- Implement meaningful notifications.
- Keep code modular.
- Document complex logic.
- Maintain migrations.
- Run available tests and builds.

Avoid:

- Giant components.

- Duplicated business logic.

- Hardcoded credentials.

- Fake success messages.

- Unnecessary dependencies.

- Unverified provider assumptions.

- Uncontrolled database access.

- Unnecessary full-page reloads.

- Rebuilding working modules without justification.

44. TESTING

Test:

- Authentication.

- Authorization.

- RLS.

- Project CRUD.

- Data ownership.

- Language switching.

- Input validation.

- Error handling.

- Provider failures.

- Job retries.

- Credit accounting.

- Approval permissions.

For important functionality, provide automated tests where practical.

Never claim tests passed if they were not actually run.

45. DOCUMENTATION

Keep these documents updated:

- AI_MOVIE_FACTORY_SSOT.md
- README.md
- CHANGELOG.md
- ARCHITECTURE.md
- DATABASE_SCHEMA.md
- ENVIRONMENT_VARIABLES.md
- API_DOCUMENTATION.md
- DEVELOPMENT_ROADMAP.md

Update documentation whenever a significant architectural or product decision changes.

PART 16 — PROJECT MANAGEMENT RULES FOR THE AI DEVELOPER

46. PERMANENT DEVELOPMENT BEHAVIOR

For every development task:

1. Read this SSOT.
2. Inspect the current implementation.
3. Identify the relevant module.
4. Check dependencies.
5. Determine database changes.
6. Determine security implications.
7. Implement the smallest complete change.
8. Run relevant checks.
9. Fix discovered errors.
10. Update documentation.
11. Report the actual result.

Before modifying existing functionality, understand how it is currently used.

Do not remove functionality without explicit approval.

Do not silently change database structures that existing features depend on.

Use migrations for schema changes.

47. COMPLETION REPORT

At the end of each implementation task, provide:

A. Completed

List functionality actually implemented.

B. Modified Files

List important files changed.

C. Database Changes

List migrations, tables, policies, or functions changed.

D. Verification

List tests, type checks, and builds actually executed and their results.

E. Incomplete

List anything that remains unfinished.

F. Risks

List important security, compatibility, cost, or deployment concerns.

G. Next Recommended Task

Recommend the next logical implementation task.

Never mark a task complete merely because the UI looks finished.

PART 17 — FIRST IMPLEMENTATION INSTRUCTIONS

48. IMMEDIATE TASK

Start by inspecting the existing Google AI Studio project.

Do not immediately build every module.

First implement Phase 1: SaaS Foundation.

The first deliverable must include:

1. Premium cinematic application shell.
2. Responsive sidebar and top navigation.
3. Indonesian as default language.
4. English language switching.
5. Supabase authentication.
6. Protected application routes.
7. User profile.
8. Dashboard.
9. Project management.
10. Dynamic project creation.

11. Project editing and deletion.

12. Initial database migrations.

13. Ownership-based RLS.

14. Setup documentation.

15. GitHub-ready project structure.

Do not implement paid AI video generation in Phase 1.

Do not implement fake payment or credit transactions.

Use realistic demo data only when clearly labeled and only where necessary for a disconnected preview.

The application must not falsely report database or authentication success.

49. FIRST PROJECT CONFIGURATION FIELDS

The project creation interface must include:

- Project name.

- Description.

- Genre.

- Custom genre.

- Production format.

- Number of seasons.

- Number of episodes.

- Episode duration.

- Visual style.

- Aspect ratio.

- Output language.

- Production mode.

- Enabled modules.

- Enabled AI agents.

- Human approval settings.

- Credit budget.

Use sensible defaults but allow users to customize their projects.

The form must be easy to use and should not overwhelm users with unnecessary settings at once. Group advanced settings logically.

50. FIRST DATABASE MIGRATIONS

Implement the initial database foundation:

profiles

- id.

- display_name.

- avatar_url.

- preferred_language.

- created_at.

- updated_at.

projects

- id.
- user_id.
- name.
- description.
- genre.
- custom_genre.
- production_format.
- status.
- created_at.
- updated_at.

project_settings

- id.
- project_id.
- settings JSONB or an appropriately normalized structure.
- created_at.
- updated_at.

Add suitable foreign keys, indexes, constraints, and RLS policies.

Use Supabase Auth as the identity source.

Ensure profile and project creation are correctly linked to the authenticated user.

Use migrations rather than relying on undocumented manual database modifications.

PART 18 — FINAL PRODUCT DEFINITION

51. WHAT AI MOVIE FACTORY MUST BECOME

AI Movie Factory must evolve into a complete modular AI film production ecosystem.

It should allow a creator to:

1. Start a project.
2. Define its universe.
3. Create characters.
4. Develop a story.
5. Write scripts.
6. Organize episodes.
7. Break episodes into scenes.
8. Create storyboards.
9. Generate visual assets.

10. Generate video.

11. Generate voices and audio.

12. Coordinate AI agents.

13. Manage production tasks.

14. Review quality and continuity.

15. Approve and revise scenes.

16. Manage credits and costs.

17. Export finished production assets.

All of this must work within one coherent project environment.

The platform should be powerful enough for production studios while remaining accessible to individual creators.

52. FINAL DIRECTIVE

Build AI Movie Factory as a long-term commercial SaaS product.

Do not treat it as a one-time demo.

Do not reduce it to a generic chatbot.

Do not restrict it to one genre.

Do not lock it to one AI provider.

Do not sacrifice security for development speed.

Do not sacrifice maintainability for visual appearance.

Do not implement expensive infrastructure before it is needed.

Do not pretend planned features are already functional.

The SSOT is the permanent product and engineering reference.

Your immediate responsibility is to inspect the current project and implement Phase 1 completely.

When Phase 1 is complete, report the actual implementation status and recommend the next phase.

END OF MASTER SSOT DOCUMENT